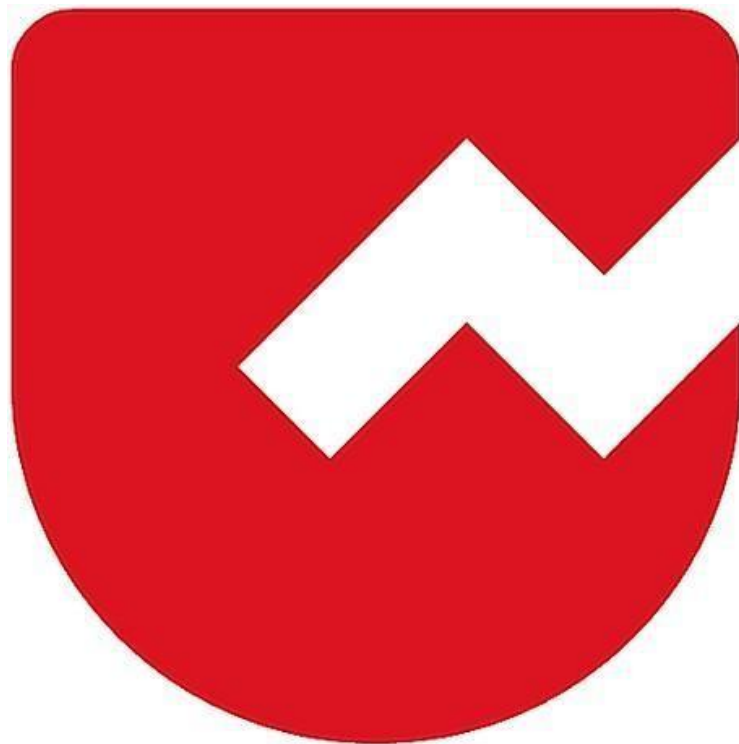




Zoho Schools for Graduate Studies



Notes

INHERITANCE

WHAT IS INHERITANCE:

- It is a property by which one class(sub-class) acquires the **inheritable members** of another class(super-class).

```
public class Animal { // base class or parent class
    int lifespan;

    void eat() {
        System.out.println("Animal eats");
    }
}
```

```
public class Dog extends Animal {
    // Dog has no member inside the class
    // but it inherit members from Animal
    // Hence the members lifespan & eat() used in both Animal & Dog class
    // to avoid code replication
}
```

```
public class InheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        System.out.println(d.lifespan);
        d.eat();
    }
}
```

NOTE:

- Subclass will not inherit all members from superclass.
- Subclass use “**extends**” keyword to inherit from its superclass.

WHAT ARE THE ADVANTAGES OF INHERITANCE :

- **Code Reusability** or to avoid code redundancy or duplication or replication.
- To obtain **package access control**/package private access control/default access control.

WHEN TO PREFER INHERITANCE? OR WHY INHERITANCE?

- Inheritance is preferred whenever a class wants to reuse or share **certain members**(can be attributes or behaviours) of another class.

SUBCLASS:

- The class that is **inheriting** is referred as subclass or child class or derived class.

SUPERCLASS:

- The class that is **inherited** is referred as superclass or base class or parent class.

```
public class Animal { // base class or parent class
    int lifespan;

    void eat() {
        System.out.println("Animal eats");
    }
}
```

```
public class Dog extends Animal { //Derived class or child class or sub class
    void bark() {

        System.out.println("Dog barks lol.. lol.. lol..");
    }

    void eat() { // method overriding
        // want to capture specific behavior of Dog
    }
}
```

```

        super.eat();

        System.out.println("Dog eats milk, meat, biscuit...");
    }

    void wagTail() {

        System.out.println("Dog wags its Tail...");

    }
}

```

```

public class InheritanceDemo {

    public static void main(String[] args) {

        Dog d = new Dog();

        System.out.println(d.lifespan);

        d.eat();

    }

}

```

NOTE:

- The **super keyword** is used to inherit the parent class members.
- It is also used by the subclass to access superclass methods or constructors.

QUESTION:

- ✓ If Animal's eat() is inherited to Dog? — Yes.
- ✓ How many members for Dog cls? — 5

WHEN METHOD OVERRIDING POSSIBLE?

- Method overriding is possible if and only if the super class method is inheritable

METHOD OVERLOADING VS METHOD OVERRIDING

1. Method Overloading

Two or more methods in the same class with the same name but different parameter lists.

Example:

```
Aclass Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void eat(String food) { // Overloading with parameter  
        System.out.println("Dog eats milk, meat, biscuit...");  
    }  
}  
  
public class OverloadDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat(); // Dog eats food  
        d.eat("bone"); // Dog eats bone  
    }  
}
```

2. Method Overriding

Subclass provides its own implementation of a method that is already defined in the superclass.

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
class Dog extends Animal {  
    void eat() { // Overriding parent class method  
        super.eat(); // call parent method  
        System.out.println("Dog eats milk, meat, biscuit...");  
    }  
}
```

```
public class OverrideDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        // Animal eats  
        // Dog eats milk, meat, biscuit...  
    }  
}
```

HOW IS THE METHOD OVERRIDING OBTAINED?

LEGAL RULES FOR METHOD OVERRIDING :

1. Overridden method must share the same method signature.
2. Overridden method can retain the same access level or can have a broader access level.
3. Overridden methods can have the same return type or any subtype of the superclass return type (covariant return type).

```
public class Animal {  
    int lifespan;  
  
    public Number eat() {  
        System.out.println("Animals eat");  
        return 57;  
    }  
}
```

```
public class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks loud... loud... loud...");  
    }  
  
    void wagTail() {  
        System.out.println("Dog wags its tail...");  
    }  
}
```

```
public Integer eat() {  
    System.out.println("Dog eats milk, meat, biscuit...");  
    return 20;  
}  
}
```

```
public class InheritanceDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
    }  
}
```

```
        int age = d.eat();
        System.out.println(age);
    }
}
```

Output:

```
Dog eats milk, meat, biscuit...
20
```

NOTES:

- Order of access control: private, default, protected, public.

HOW PROTECTED WORKS IN PARENT VS CHILD:

Protected in Parent Class

```
class Animal {
    protected void eat() {           // protected method
        System.out.println("Animal eats");
    }
}
```

```
public class ProtectedParentDemo {
    public static void main(String[] args) {
        Animal a = new Animal();
        a.eat(); // Allowed (same package)
    }
}
```

Protected in Child Class (Access through Inheritance)

```
class Animal {  
    protected void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
  
    void show() {  
        eat(); // Dog can access protected method from Animal  
        bark();  
    }  
}  
  
public class ProtectedChildDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.show(); // Accessing protected method through child class  
    }  
}
```

protected allows access:

- Within the same package.
- In subclasses (even if in different package).
- But not accessible directly by unrelated classes in another package.